RTDL: A Restricted-Python DSL for Repurposed RT

Anonymous Author(s)

Abstract

Ray-tracing (RT) hardware increasingly executes non-rendering workloads in which a geometric hit denotes an application candidate, record, neighbor, or aggregate contribution. Such meanings impose obligations on coverage, identity, repeated delivery, termination, and reduction. These obligations cross source callbacks, data and geometry interpretation, physical traversal, and result return; local stage legality and machine types do not capture the whole relation.

We present RTDL, a restricted-Python domain-specific language and compiler for bounded repurposed RT computations. Authors declare immutable records and typed role functions. The compiler constructs canonical Callback IR, checks role effects, resources, semantic and physical contracts, and supported result routes, then lowers through a deterministic ABI and generated Numba leaves into trusted topology-specific CUDA/OptiX wrappers. Prepared execution binds the admitted program, target, and runtime identities. For fixed families, a result obligation changes concrete compiler decisions: complete-relation routes reject unsupported early termination, logical all-hit acceptance generates a payload update followed by physical intersection rejection, and exact callback IR selects guarded standard-route specialization.

The implementation exposes two stable constructors and a bounded corpus over triangle, custom-AABB, curve, and sphere leaves. A historical audit covers nine project-authored V4 mappings. A frozen public-PyOptiX comparison covered selected stages from three mappings (four operation units); all twelve complete, first-execute, and prepared observations were slower (1.080–75.533×). Separately, two exact standard routes were 1.077–1.175× Direct when prepared, while post-import medians were 1.560–1.837× a strong PyOptiX baseline, with a 2.377× worst block. A sealed composition and a finite independent checker provide bounded compiler evidence. RTDL does not infer profitable mappings, accept arbitrary Python, synthesize arbitrary topologies, prove application semantics, or establish easier authoring or broad practical performance.

CCS Concepts: • **Software and its engineering → Compilers**.

Keywords: ray tracing, domain-specific languages, restricted Python, GPU compilation

ACM Reference Format:

Anonymous Author(s). 2027. RTDL: A Restricted-Python DSL for Repurposed RT. In *Proceedings of IEEE/ACM International Symposium on Code Generation and Optimization (CGO '27).* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Dedicated RT traversal is now used for particle tracking, graph processing, spatial indexing and joins, neighbor search, clustering, database processing, and N-body simulation [6, 7, 19, 21, 22, 29, 34, 35, 37]. These systems show that a bounding-volume hierarchy can accelerate much more than image synthesis. They also expose a recurring programming problem. A hardware hit may be only a candidate, one logical row, a neighbor, or one contribution to a checked aggregate. The corresponding requirements for filtering, logical identity, repeated delivery, early termination, and reduction differ by result.

Successful applications already solve concrete instances. RayJoin records all accepted intersections and physically ignores each one so traversal continues; RayDB prevents duplicate delivery of a primitive from corrupting aggregation; RTNN uses different maintenance rules for bounded range and nearest-neighbor results; LibRTS offers spatial-query handlers for counting and collection [6, 7, 29, 37]. These are prior solutions and motivate, rather than constitute, our contribution. The language question is how a programmable interface can represent selected obligations so that they constrain both admissible callbacks and generated execution.

Modern RT systems already provide substantial machinery. OptiX exposes the callback protocol; PyOptiX and SlangPy bring RT construction to Python; OWL and Luisa automate pipeline construction; DXR payload qualifiers and Slang capabilities enforce stage and target constraints [15, 20, 25–27, 30, 32, 36]. Rendering languages also connect domain results to renderer-owned execution; for example, Open Shading Language separates closure construction from integrator evaluation [1]. Our claim is not that rendering lacks protocols or that prior systems cannot express the same guarantee. We ask a narrower compiler question:

> For an author-selected RT mapping, how can a restricted high-level language express bounded non-rendering computation while making selected result obligations constrain callbacks, data interpretation, traversal actions, and result return?

RTDL answers for a bounded subset. Authors write immutable records, restricted role functions, and manifests that name data, resources, and result contracts. The compiler builds typed Callback IR, checks cross-role and result-route constraints, generates leaf code and trusted topology

wrappers, and binds the accepted executable to a prepared runtime. The application still owns the RT formulation, geometric encoding, domain predicates, and semantic oracle.

This paper makes four contributions:

1. We design a restricted-Python DSL and programming model for bounded repurposed RT. It exposes typed records and role-indexed effects while keeping raw PTX, SBT, pipeline, and traversal operations compiler-owned.
2. We develop typed Callback IR and whole-protocol admission that relate roles, result obligations, semantic and physical data contracts, resources, continuation, and executable identity. This is a fixed-family design, not automatic inference or a soundness proof.
3. We implement a deterministic ABI, generated Numba leaves, trusted topology-specific wrappers and exact-IR specialization, plus a prepared identity-checked runtime. The lowerers and specializers remain in the TCB.
4. We reconstruct nine project-authored V4 mappings and evaluate bounded mutation, composition, target checking, lifecycle, and performance. A selected-stage 3/9 comparison is adverse at all twelve endpoints; two separate exact-route experiments bound prepared overhead near Direct. Receipt, startup, coverage, generality, and authoring limits remain explicit.

2 RTDL by Example and Programming Model

Figure 1 shows an abridged version of the supported built-in-triangle all-hit count callback used by the standard library. The full program also defines `make_ray` and `miss`; those roles are omitted only from the figure. The front end parses this text without importing or executing it as Python.

The author supplies the callback source, manifest, records, constants, geometry and data bindings, result contract, and an application oracle. The manifest also separates *static input*, such as geometry and primitive metadata prepared once, from *batch input*, such as query rays supplied repeatedly. The compiler owns parsing, verification, role/effect IR, ABI flattening, leaf code, trusted wrapper selection, PTX composition, program/SBT binding, status handling, and prepared lifecycle. Users cannot inject raw traversal calls, arbitrary PTX, native callback pointers, or self-declared physical authority through the stable surface.

Compilation follows:

source + manifest → typed Callback IR → whole-protocol admission → ABI/Numba leaves → trusted wrapper/PTX → bind/prepare/execute.

The language surface is broader than the two stable closed constructors, but not every verified callback/topology combination has an executable lowerer. Unsupported combinations fail closed rather than falling back to arbitrary Python or handwritten target code.

```
@optix.payload
class CountPayload:
    count: u64

@optix.output
class CountOutput:
    count: u64

@optix.program(
    payload=CountPayload, output=CountOutput,
    attributes=(), max_trace_depth=1,
    max_callable_depth=0)
class TrianglePerRayCount:
    # make_ray and miss omitted here
    @optix.any_hit
    def any_hit(hit: TriangleHit,
                payload: CountPayload) -> AnyHitEffect:
        updated = CountPayload(
            count=payload.count + 1)
        return optix.accept_continue(
            payload=updated)

    @optix.finalize
    def finalize(payload: CountPayload) -> CountOutput:
        return optix.output(value=CountOutput(
            count=payload.count))
```

Figure 1. Abridged real RTDL source for a supported all-hit count. The manifest, ray construction, and miss role are omitted for space; the figure is not a standalone executable.

Table 1 grounds the language problem in three prior systems that repurpose RT hardware. Each already implements its listed solution. RTDL targets the narrower compiler relation between a declared obligation and its own accepted/generated route; it does not take ownership of the underlying application predicate or algorithm.

3 IR and Whole-Protocol Checking

3.1 Why callback typing is insufficient

A ray callback runs inside a protocol distributed across host and device code. Its legal effects depend on its role: an any-hit callback may ignore or terminate an intersection, whereas a miss callback cannot. Its payload fields must agree with every producer and consumer. Its program group must match the acceleration-structure leaf kind. The launch must occur only after module, pipeline, shader-binding-table, geometry, and output state have been prepared. Finally, the code checked by the compiler must be the code loaded by the runtime.

These are related to typestate, interface automata, and session-style protocol reasoning [3, 13, 33], but the FFI and generated-code boundary adds an identity problem familiar from linking and foreign interfaces [5, 28]. Table 2 makes the additional result-related decisions concrete. Each row is an implemented fixed-family rule, not a theorem for arbitrary RT programs.

Table 1. Recurring result obligations in prior repurposed-RT systems. These are motivation and prior solutions, not RTDL inventions.

System	Result obligation	Existing application solution	RTDL's bounded compiler target
RTNN [37]	Range-K may stop at K; nearest-K needs ranked candidates	Intersection filtering plus result-specific bounded storage or priority queue	Represent the selected continuation/result contract; do not infer distance correctness or search coverage
RayJoin [6]	Complete logical intersections despite AABB false positives	Exact predicate, result recording, then physical intersection ignore to continue	Relate logical event acceptance to generated traversal action and bounded publication
RayDB [29]	A record delivered by multiple rays must not corrupt aggregation	Primitive identity plus first-delivery atomic flag before aggregation	Represent identity/delivery/reducer obligations for supported routes; do not infer SQL bag semantics

Table 2. Implemented result-route relations and their evidence boundaries.

Observable result obligation	Not decided by stage or machine typing	RTDL constraint or transformation	Evidence boundary
Complete bounded relation	Whether locally legal `TERMINATE` or `IGNORE` fits this route	The family verifier rejects the route unless returns are `ACCEPT_CONTINUE`; capacity failure publishes no partial relation	Source rule and existing CPU checks; not a general enumeration theory
Triangle all-hit count	Whether logical acceptance should physically accept an intersection	The generator emits a payload update, then calls `optixIgnoreIntersection()`; require single any-hit delivery and checked overflow	Generic and specialized lowering source; OptiX mechanisms are established, not invented here
Standard count specialization	Whether same-shape `+1` and `+2` callbacks may share a fixed intrinsic	The generator selects the intrinsic only under its exact-IR guard; `+2` selects the general leaf path	Existing source-to-wrapper test replay; no GPU equivalence or general optimization theorem
Relation application ID	Whether an attribute denotes physical index or application ID despite equal `u32` width	Admission rejects a mismatch between trusted schema and compiled-contract projection	Existing field mutation and mock; application intent is not inferred

3.2 A concrete semantic-ABI failure

Consider two acceleration-structure primitives at physical positions 0 and 1 whose application identifiers are 10 and 20. Queries 100 and 101 should emit the pairs $(100, 10)$ and $(101, 20)$. If an intersection producer instead writes the physical position into attribute slot zero while the consumer interprets that slot as an application identifier, the apparent rows are $(100, 0)$ and $(101, 1)$. Every value remains a valid `u32`; width, slot, and row layout need not expose the semantic substitution. This is an illustrative witness, not a retained execution of the defective route.

The defect is not a machine-type error but a disagreement over what the bits mean. In RTDL, a trusted bounded-relation schema declares that attribute slot zero denotes an application identifier; the compiled relation contract carries that row source into a separately derived target projection. Admission compares the two compiler representations. Existing tests mutate only this fact and obtain `CP002_ATTRIBUTE_ABI_OWNERSHIP_MISMATCH`; with native initialization overlap disabled, an integrated projection mutation is rejected before the native-library loader is called. This proves check liveness, not automatic inference of application meaning or end-to-end execution of the illustrative defect. Both derivations remain in one compiler TCB, and a wrong but internally coherent trusted schema can pass.

3.3 Failure model

We distinguish four outcomes. *Reject* means admission fails before a launch. *Status failure* means execution reports failure and output is not published. *Wrong output* means a worker reports success but an independent oracle disagrees. *Unverified output* means a value exists without the required receipt or oracle evidence. The distinction prevents

a status gate from being described as numerical correctness and prevents a generated plan from being described as an executable lowering.

A focused mock reproduced an unrepaired provider double fault: secondary bind or close failure can replace the primary exception and lose cleanup retry ownership; the mock returned no public result. A native-fork probe accepted a public call through a mock native implementation after bypassing the registered Python at-fork hook; it did not execute GPU work. Neither defect was observed in retained successful GPU workers, and inherited owners after such forks remain unsupported.

3.4 Typed callback representation

The source language is restricted Python, not arbitrary Python. Source is parsed through an AST allowlist and is not imported or executed for discovery. The front end resolves declared immutable records, frozen constants, same-unit acyclic helpers, and selected intrinsics, then produces canonical typed IR. Deserialization reconstructs typed objects and reruns verification; serialized IR is not itself an authority. Table 3 summarizes the representation.

The current resource contract admits at most 32 payload slots and eight attribute slots, trace depth one, callable depth zero, bounded helper depth, and statically bounded iteration. The numeric policy requires explicit binary32 behavior and fail-closed handling of prohibited integer overflow and nonfinite effects. These are language and resource checks; they do not prove that a chosen ray encoding computes the intended application function.

3.5 Role-indexed effects

Seven language roles participate in an admitted unit. `bounds` runs during preparation; `make_ray` and `finalize` are language leaves invoked by a trusted ray-generation wrapper; and `intersection`, `any_hit`, `closest_hit`, and `miss` map to target-stage behavior. User code cannot invoke raw traversal. Instead, each role returns one typed effect from a role-specific set: bounds returns an AABB, ray construction returns a trace request, intersection returns hit or no-hit, hit callbacks return payload plus continue/ignore/terminate control, miss returns payload, and finalize returns an output.

Topology makes these requirements relational. Custom AABBs require bounds, ray construction, intersection, miss, finalize, and at least one hit role. Built-in triangles forbid bounds and intersection because hardware owns them, but retain ray construction, miss, finalize, and at least one hit role. An any-hit role also requires an explicit delivery and continuation contract; the presence of that declaration does not prove order independence for arbitrary callback logic.

Role legality is necessary but not sufficient for a fixed result route. The general any-hit role admits continue, ignore, and terminate effects. The current complete bounded-relation verifier instead collects all return effects and requires only logical acceptance with continued traversal; it defines no filtering or early-termination meaning for that route. Thus a role-valid effect can be rejected because it conflicts with the route's output obligation. This is an implemented family restriction, not a claim that all complete-enumeration designs must reject filtering.

Logical effects also need not map one-to-one to physical intersection actions. In the triangle all-hit route, the logical continue effect first updates the payload and then physically ignores the intersection, preserving later events instead of shrinking traversal's accepted interval. Native geometry separately requests one any-hit delivery per primitive, and the reduction checks overflow. These are established OptiX mechanisms; RTDL's contribution is to make their required combination part of the trusted implementation of a fixed result contract, not to invent all-hit traversal.

3.6 Fail-closed judgments

Expression checking derives $\Gamma \vdash e : \tau$. Statement checking also returns the output environment, all-paths-return bit, conservative static work, and observed effects:

$$\Gamma; r \vdash s \Rightarrow \langle \Gamma', q, n, \epsilon \rangle. \tag{1}$$

For each role r, verification enforces its exact signature and

$$M \vdash F_r : \epsilon \quad \text{with} \quad \epsilon \subseteq \mathsf{AllowedEffects}(r). \tag{2}$$

Names are single-assignment except for type-preserving updates inside static loops; loop locals cannot escape; both non-returning branches define the same typed names; and every role/helper path returns. Whole-unit checking validates schema/source identity, acyclic records and helpers, numeric and resource budgets, role cardinality, topology, linkage, and required external physical authority:

$$\mathcal{G}; \mathcal{R} \vdash Q \Downarrow \mathsf{Verified}(h_{\mathrm{IR}}, h_{\mathrm{effects}}). \tag{3}$$

Here $\mathcal{G}$ is an out-of-program geometry authority and $\mathcal{R}$ is the compiler-owned role topology. The judgment is explanatory notation for implemented checks, not a mechanized soundness theorem.

Admission is conjunctive. A route is accepted only if its schema is valid, its roles permit all effects, its semantic and physical ABIs agree, a trusted lowerer exists for its topology, the generated executable identity is bound, and the runtime lifecycle can reach the prepared state. Failure of any one predicate rejects the route; there is no "best effort" projection into a nearby executable.

3.7 Layered whole-protocol admission

Table 4 identifies where each obligation originates. This separation matters because target evidence is regenerated

Table 3. Implemented bounded representation. Rejected forms have no fallback lowering.

Form	Accepted subset	Rejected boundary
Types	Booleans; fixed-width integers and floats; 2–4-lane vectors; tuples; acyclic immutable records; checked read-only views	Raw pointers, mutable containers, opaque objects, and recursive records
Expressions	Typed names and literals, record fields, checked indexing, arithmetic, comparisons, bit operations, constructors, selected pure intrinsics, and bounded helper calls	Reflection, dynamic dispatch, allocation, foreign calls, and unknown attributes
Statements	Typed binding, loop-local type-preserving update, two-sided conditionals, statically bounded loops, role-effect return, and helper-value return	Dynamic loops, recursion, exceptions, generators, async control, and not-all-paths-return
Manifest	Payload/output records, machine attributes, constants, numeric policy, resource budget, topology, continuation, and linkage	Self-minted physical authority, unreviewed production linkage, and unsupported budgets
Program	Canonical source and IR identities, records, functions, manifest, and separately derived effect digest	Unknown fields, malformed schema, stale identities, or unverified deserialization

within one trusted implementation; it is not an independent proof of compiler correctness.

4 Code Generation and Runtime

4.1 Leaves, wrappers, and specialization

Verification produces one role-indexed ABI containing flattened inputs, outputs, effect tags, status tags, and identities. For the general route, the compiler emits deterministic Python source for each reachable role and helper, then invokes an isolated Numba compiler process to produce target-specific PTX device functions. The isolated child receives compiler-generated source rather than user modules and does not discover the active CUDA device. This leaf step does not build an OptiX pipeline.

A trusted topology wrapper composes the leaves with compiler-owned raygen, intersection, hit, and miss mechanics. It checks returned tags before issuing payload writes, `optixIgnoreIntersection()`, or termination. Thus the user effect `accept_continue` is a logical action; its physical action depends on the admitted route. Wrapper source, leaf PTX, ABI, provider symbols, and physical schema are hashed into the executable identity.

The evaluated standard routes also use trusted specializations guarded by exact callback IR and reducer identity. For the standard count program in Figure 1, the specialized entry implements the known increment-and-continue behavior directly; the general path obtains the same declared behavior through the leaf ABI and wrapper. Bounded relation similarly fuses intersection and row emission. These specializations replace generic leaf calls and per-leaf tag interpretation at selected roles, but retain static schema, ABI, identity, overflow, capacity, status, and supplied expected-output checks. An exact-IR guard selects trusted code; it is not a proof that the specialization preserves source semantics.

Selection depends on program identity, not just role and ABI shape. An existing compiler test changes the standard increment from one to two. The program remains front-end legal and has the same role shape, but its IR identity changes and the generator exits the fixed count intrinsic for the general leaf path. This source-to-wrapper check establishes one selection boundary; it neither executes the modified program on a GPU nor proves arbitrary specialization correctness. The recognizer, wrapper, and specialized lowerer remain in the trusted computing base (TCB).

4.2 Plan versus executable lowering

The canonical planner normalizes the admitted schema and emits a declarative plan. That plan is intentionally marked non-executable. It does not synthesize arbitrary CUDA or OptiX callbacks. Instead, an accepted route names a compiler-owned, topology-specific lowerer and wrapper. The lowerer emits source and PTX, binds native symbols, constructs the program groups and shader-binding table, and records their identities. This design supplies a common admission surface while keeping concrete backend knowledge visible in the TCB.

Shared schema and lifecycle machinery do not make device code generic. The prospective sphere route in Section 6.3 required about 2,635 lines of topology-specific code and 28 modified compiler lines. It shows that a trusted route can reuse three sealed shared files, not automatic topology lowering.

4.3 Executable identity and lifecycle

The system binds hashes and shape metadata for generated source, PTX, ABI, native provider symbols, and the public route. The exact app-free native runtime may be loaded and warmed before route admission, including concurrently

Table 4. Authorities compared by whole-protocol admission.

Obligation	Declaration authority	Target/runtime evidence	Exact limit
Role/effect	Verified role topology and returned effects	Reverified compiled callback ABI variants	Common compiler TCB; no arbitrary effect inference
Semantic ABI	Trusted family schema and fixed-route nominal row-source declaration plus machine types	Compiled relation contract and callback ABI	Same compiler TCB; meaning is declared, not inferred; generic `u32` has no nominal meaning
Physical ABI	Typed plan for geometry, buffers, hit channel, result, and reducer	Rebuilt target facts and bound storage	Exact-plan admission, not arbitrary physical synthesis
Continuation	Completeness/capacity policy and required status-before-use order	Status vocabulary, compiled continuation, runtime status gate	Bounded supported modes; not a confluence theorem
Identity/lifecycle	Materializer-selected source/PTX/provider identity and state machine	Loaded bytes, prepared objects, execution status, output digest, and optional receipt	Hash equality is identity coherence, not semantic correctness

with AOT artifact verification. Admission still gates acceptance of a materialized or loaded route and its subsequent per-route preparation. The admitted route lifecycle is:

$$\text{static verify} \rightarrow \text{materialize/bind} \rightarrow \text{prepare} \rightarrow \text{execute/status gate} \rightarrow \text{output} \rightarrow \text{receipt}.$$

The materialized-program interface validates execution identity, native status, output digest, and traversal receipt before returning `ProtocolExecutionResult`. The measured AOT path instead returns `RTDLExecutionResult`: it synchronously rejects native and compact-status failures and any supplied expected-output mismatch, but an ordinary fast result may omit a digest and detailed receipt. The worker checks value and digest after prepared return; for first result this is after the prepared timer but inside the endpoint. It does not rehash disk files per launch. The preregistered per-execution receipt requirement is unmet: 4,096 timed A executions have only 32 separate post-loop diagnostic executions, one detailed receipt per A worker.

5 V4 Applications and Case Studies

RTDL was developed against repurposed-RT applications rather than rendering shaders alone. Table 5 reconstructs nine project-authored V4 mappings to algorithms credited to prior systems. RTDL contributes restricted expression, checking, generation, and composition of selected parts, not the application algorithm or geometric mapping. E1 means an exact historical application-source hash and callback-consumer binding survive. E2 means a frozen audit recorded the path and structural properties, but an independently retained historical per-file identity and the original 10.8-MB archive do not; recovered current entry points do not repair that custody gap. E2 is mapping evidence, not current runnability.

A frozen AST audit found no selected raw OptiX, CUDA, or PTX assembly tokens in the nine application files. This supports a structural shift in physical-owner preparation, not developer-time or LOC savings. Runtime loading passed through registered V4 interfaces for eight applications; RayDB is the exception, and trusted C++/CUDA/OptiX partners remain. The stable facade has two constructors, covers 2/6 OptiX build-input values and 2/4 leaf kinds, while the bounded corpus covers 4/6 values and four leaf kinds. Only one callback template is presently authorable, no independent user session was measured, and every new topology still requires a trusted lowerer.

6 Evaluation

We organize existing evidence around five questions. **RQ1** asks whether DSL and IR facts change admission, generation, or specialization. **RQ2** asks what historical mappings reuse and what remains application-owned. **RQ3** asks what a sealed extension reuses and what remains specific to a topology. **RQ4** asks what finite target properties an independent checker can validate. **RQ5** asks the runtime cost of exact and application paths. Evidence types remain separate: source traces and component tests are not GPU equivalence tests, historical mappings are not current runnable examples, finite mutations are not a soundness proof, and measured routes are not arbitrary-program lowering.

6.1 Earlier liveness and residual evidence

For RQ1, the role-level any-hit set permits continue, ignore, and terminate, while the complete bounded-relation route accepts only continue returns. The triangle lowerer supplies the complementary transformation evidence: it records an accepted event before physically ignoring the intersection to preserve later events. These are source-traced implemented rules; this paper did not execute a deliberately terminating complete-relation program on the GPU.

Also for RQ1, we replayed an existing compiler test that changes the standard count update from +1 to +2. Front-end verification still succeeds, the callback IR identity changes, and wrapper generation switches from the fixed count intrinsic to the general leaf path. The test exercises parsing, IR

Untrusted program and data → restricted-Python front end → shared schema, role/effect, ABI, and topology admission → non-executable canonical plan

Trusted topology route → topology-specific lowerer and wrapper → generated source/PTX and provider symbols → executable-identity binding → OptiX build, prepare, and launch

Measured AOT prepared execution → native status gate → compact status gate → supplied expected-output check, when present → public return (prepared steady timer ends)

Experiment validation → returned value → worker output-and-digest oracle → accepted sample or worker success (first-result endpoint ends)

Separate diagnostic and evidence paths → post-loop diagnostic execution and one retained detailed receipt per A worker → offline checker over generated source, PTX, ABI, symbols, and receipts; it does not import the RTDL compiler.

Figure 2. Architecture and trust boundary. Shared admission does not replace the topology-specific lowerer; that lowerer remains in the TCB.

Table 5. Historical V4 mappings. Authors select the RT formulation and own the listed application semantics; E1/E2 are custody levels defined in text.

Application	Mapping and author-owned semantics	RTDL route and exact boundary
Particle [34]	Tetrahedral closest-face transition; cell encoding, transition, and I/O meaning	E1: built-in triangles, restricted closest-hit update, callback lowering, prepared lifecycle
Triangle [35]	RT-1A2/RT-2A1 selection; graph orientation and count meaning	E1: all-hit callback, checked capacity/status, segmented reduction; measured scalar is one algorithm stage
RayDB [29]	Partitioned grouped-I64 aggregate; relation, keys, partitions, and SQL meaning	E2: bounded emission and grouped reduction; sole private-loader exception
LibRTS [7]	AABB point/range containment; predicate, schema, and count/collect meaning	E1: typed custom-AABB relation and capacity/receipt checks; LibRTS is strong prior abstraction
X-HD [8]	Directed max-of-nearest witness; Hausdorff semantics and output	E2: cell-MBR traversal, checked state, tie-breaking, and reduction
RTNN [37]	Multiround distance-window top-k; metric, K, bounds, and ordering	E2: round/refit lifecycle and bounded selection; metric-specific traversal stays trusted
RT-DBSCAN [22]	Radius graph/components; radius, min-points, and clustering meaning	E2: bounded edges and grouped continuation; not every stage uses RT cores
RayJoin [6]	Six-batch planar overlay; schedule, predicates, and six full result tables	E2: typed producer/reducer route; recovered adapter returns summaries, not the registered full tables
RT-BarnesHut [21]	Aggregate hierarchy; tree, acceptance rule, and force law	E2: hierarchy/GAS ABI and frontier checks; tree traversal remains a trusted partner

verification, contract/ABI construction, and wrapper generation. It does not execute either generated path on a GPU or establish semantic preservation for other optimizations.

For two fixed contracts, 19 single-leaf mutations exercised 19/19 expected gates: seven role/effect, one semantic-ABI, seven physical-ABI, two continuation, and two identity mutations. This denominator covers only those contracts and mutations and is not pooled with later experiments.

With native-initialization overlap disabled, corrupting each target projection produced the expected sole reason before the native-library loader was called. For semantic ABI, this is mutation sensitivity, not an execution of the illustrative wrong-row route. Historical PyOptiX/OWL-style diagnostics lack independently verifiable execution records here and are not evidence about those systems' guarantees [25, 26]. The 19/19 result is neither soundness nor prevalence evidence.

6.2 Historical application evidence

For RQ2, the frozen source audit behind Table 5 classified all nine V4 mappings as a structural integration-responsibility shift and found registered-interface runtime loading in eight. It measured neither authoring time nor developer productivity. Three exact historical app/callback consumer identities remain independently recoverable. Current entry points have been recovered for all nine, but the other six retain only archive-level historical audit records. We therefore use the portfolio to show bounded mapping and composition diversity, not historical byte custody, current installability, or unseen-app success.

A separate historical RTX 4000 Ada authority reported 464 exact, behaviorally true-OptiX workers and 34 V2-direct/V4 rows. Its row-local median criterion split 16 pass and 18 fail; 95% confidence-interval classification gave 11 clear V4 wins, 10 clear losses, and 13 uncertain. The raw archive is not part of the current nine-member submission artifact and was not rerun here. These mixed results are retained to prevent an inference that nine mappings imply broad performance superiority. They are not a PyOptiX comparison and are not pooled with the final two-task measurements below.

6.3 Sealed-snapshot composition exam

For RQ3, before a public randomness pulse, we fixed an author-defined ten-row challenge: seven built-in four-role and three custom six-role rows. All used supported primitive families and returned one value per query: six produced counts, and four produced Booleans by terminating on the first accepted hit. The curve any-hit-terminate row remained eligible but unselected. The pulse selected builtin_sphere::any_hit_count_continue_u64_per_query.

Private custody identifies the sealed source. No bytes changed in three frozen schema, admission, and lifecycle files. The route made two OptiX launches and passed 12/12 rational-oracle rows, showing reuse of admission, identity, and lifecycle for this composition. It is not an unbiased application: authors defined the domain, rows recombined known ingredients, and the selected route required substantial topology-specific code.

6.4 Independent finite checker

For RQ4, the offline checker imports no RTDL module or compiler projection. It reads generated source, PTX/ABI metadata, symbols, and receipts. Across three route groups and four modes, it applied five property classes 20 times using 15 unique mutations, checking selected target-code patterns and ordering constraints.

This checker is finite, not a verifier for arbitrary device control flow. During review, an out-of-authority in-memory probe inserted an unconditional early return and still passed selected effect/status helper checks. The result therefore supports specialized target-structure validation only. General control-flow, numerical, and refinement soundness remain open; tools such as GPUVerify illustrate the substantially stronger proof obligation [2].

6.5 Performance methodology

For RQ5, we measured two exact tasks. *Relation* uses 4,096 objects and 4,096 queries, returns 4,096 canonical u32 pair rows, and performs two true OptiX traversals. *Triangle* uses 16,384 primitives and 16,384 queries, performs one true traversal, and returns the checked scalar 65,530.

Five arms isolate different boundaries:

- A current RTDL ahead-of-time public path at measured source M;
- B idiomatic pinned PyOptiX-compatible baseline;
- C strong PyOptiX-compatible baseline with device continuation;
- D named Direct CUDA/OptiX reference implementation;
- E frozen predecessor control for RTDL.

We ran the same registered matrix on an RTX 4090 (Ada) and RTX 3090 (Ampere). Each generation has eight blocks, two tasks, five arms, 80 cells, and 128 steady samples per cell: 160 cells and 20,480 samples total. Ratios are medians of eight within-block ratios of integer nanoseconds; lower is better, and raw times are not compared across machines. The steady timer encloses a prepared public action through return; validation follows. Entry starts before implementation-specific imports, while post-import starts after them; both end after first-result validation. PyOptiX initializes CUDA during import, whereas RTDL does so later, so these intervals include different lifecycle work.

Table 6. Prepared public RTDL versus direct CUDA/OptiX (A/D). Ratio is lower-is-better; times are medians of worker medians.

GPU	Task	A (ms)	D (ms)	Median / max
RTX 4090	Relation	0.2810	0.2612	1.0769 / 1.0923
RTX 4090	Triangle	0.0598	0.0510	1.1751 / 1.2110
RTX 3090	Relation	0.2402	0.2198	1.0948 / 1.1188
RTX 3090	Triangle	0.0608	0.0536	1.1336 / 1.1427

The reported prepared latencies measure the disclosed exact-standard-IR triangle and relation specializations, not execution of every role through the general Numba-leaf ABI.

The registered primary criterion was prepared A/D median ≤ 1.20, with no maximum-block gate. A/C entry used median ≤ 1.20 and maximum ≤ 1.35, but the endpoint was revised after an adverse result and both first-result endpoints remain confounded. We report A/C only diagnostically. Only prepared A/E was a registered regression gate; A/E first-result values are post hoc. C/B competence used ≤ 1.05.

6.6 Prepared execution results

Table 6 reports the main observation. The public AOT path is 1.077–1.175× the direct implementation by median. The triangle maximum block on Ada is 1.211×; because no maximum A/D gate was registered, it is reported as dispersion rather than converted into a pass/fail criterion.

Across 20 observations, AOT cache-hit medians were 58.3–78.2 ms, about 1.04–2.02% of cold time. Hits still include lookup, validation, and load and are outside the prepared endpoint.

6.7 Adverse lifecycle results

Every A/C post-import median is adverse, 1.560–1.837×, with a 2.377× worst block. Because entry was revised after the adverse result, neither endpoint supports a confirmatory claim. Against E, A also regresses by about 8–22% at entry and 16–31% post-import despite favorable prepared A/E, showing material pre-steady-state protocol and Python/native lifecycle work.

Arm C outperformed B in all four formal steady comparisons (C/B ratios 0.221–0.654), establishing that B alone would be a weak baseline for this experiment. It does not establish C as globally optimal. We also recorded 1,024

Table 7. Lifecycle ratios. A/C compares current RTDL with the strong device-continuation baseline; A/E compares current with frozen predecessor control E. A/E entry and post-import rows are post hoc, non-gating diagnostics; only A/E prepared was registered. Ratios are lower-is-better.

GPU	Task	A/C entry	A/C post median	A/C post range	A/E entry	A/E post	A/E prepared
RTX 4090	Relation	0.654	1.749	1.725–1.866	1.192	1.305	0.584
RTX 4090	Triangle	0.642	1.560	1.527–1.639	1.080	1.169	0.903
RTX 3090	Relation	0.681	1.837	1.816–2.377	1.217	1.262	0.608
RTX 3090	Triangle	0.618	1.637	1.608–1.653	1.138	1.163	0.922

timer-instrumentation endpoints for A only. Paired overhead was measured only for A; no corresponding paired overhead measurement was made for B, C, D, or E.

No wrong output was observed in the final GPU samples. This statement is not a correctness proof and does not repair the detailed-receipt shortfall described in Section 4.3.

6.8 Application-route cost against public PyOptiX

A separate frozen transaction at source `c5c8be48b` compared registered complete, first-execute, and prepared endpoints on one RTX A4500 (CC 8.6, driver 550.127.05, OptiX 8.0, PyOptiX 9.1). Three of nine mappings yielded four selected-stage units: one Particle Tracking transition, RT-2A1 triangle counting, and LibRTS point/range counts. Inputs contained 3,392,530 triangles and 5,000 Particle queries, com-dblp, and 11,544,398 LibRTS boxes with 100,000 queries per operation. The other six lacked a frozen output-equivalent PyOptiX owner; RayJoin also lacked its registered six full output tables. They remain in the denominator.

Both arms consumed identical derived-input identities and returned equal public outputs. Public PyOptiX owned the full host/device route; its handwritten CUDA was precompiled to hash-bound PTX. Each endpoint used eight paired fresh-process blocks, four per arm order. Ratios are medians of within-block ratios; arm times are separate medians and need not divide to that ratio. All 192 workers finished without retry or discard.

Outputs were an ordered 5000×3 `u32` Particle matrix and `u64` counts 2,224,385, 112,729, and 105,826. Complete is not raw-domain end to end: loaders first supply an encoded Particle mesh and queries, oriented/deduplicated graph CSR, or cached LibRTS index/query columns. First execute begins after preparation; prepared adds one untimed warmup. Timed calls include status/oracle checks, digest, and evidence projection. PyOptiX PTX compilation is excluded, while V4 Particle/graph compilation is included in complete.

Physical work differs too. Particle V4 canonicalizes ignored any-hit candidates before one logical closest-hit; PyOptiX uses native closest-hit. Per graph segment, V4 rebuilds GAS and pipeline state, while PyOptiX retains pipeline objects but rebuilds GAS. LibRTS contrasts a general AoS multi-operation route with compact SoA kernels, fast-trace preference, and one stream. These are source-established differences, not causal allocations or unavoidable DSL costs. The closest result is triangle prepared at 1.379×; LibRTS's near-one complete ratios hide seconds of preparation and coexist with 38.467–39.083× prepared cost. The study establishes tested-stage functional feasibility, not speedup, end-to-end parity, broad coverage, or easier authoring.

7 Limitations and Broader Implications

Users provide bounded callback intent and data; shared RTDL machinery validates schemas, cross-role effects, ABI, identity, and lifecycle; trusted per-topology lowerers generate and bind CUDA/OptiX code. Failed status gates suppress output, while experiments may add application oracles.

This split does not make an arbitrary application correct. The author must select an RT formulation, encode geometry, declare trusted semantic meanings, provide domain predicates and reductions not covered by a fixed route, and validate the result against an independent oracle. The compiler can reject inconsistency among represented obligations, but a wrong yet internally coherent declaration can pass. Nor does verified Callback IR imply an executable path: only topologies with audited lowerers can be materialized.

The language trades expressiveness for analysis. It excludes dynamic allocation, recursion, reflection, unrestricted loops, raw pointers, and arbitrary foreign calls. Its stable facade has two constructors. The two-generation synthetic paths use exact-IR specializations; the application study also measures disclosed callback-leaf and closed native routes. No independent developer has measured authoring effort. These are design costs, not temporary omissions that the evaluation silently treats as solved.

The implementation suggests, but does not validate elsewhere, a pattern for managed-language/accelerator boundaries: use the application-visible result contract to organize callback legality, target-action interpretation, and publication; then distinguish metadata checks from trusted lowering obligations and execution-time failures. Other accelerators may expose analogous splits, but their effects, lowerers, and checker vocabularies would need independent design and evidence.

Table 8. Frozen RTX A4500 application routes. Each endpoint cell is V4/PyOptiX ms; ratio [block min–max]. Lower is better. "Complete" starts after shared preprocessing and "First" after preparation. All outputs matched; all twelve ratios are adverse to V4.

Unit	Complete	First execute	Prepared
Particle	24020.112/2405.894; 10.208 [9.559–10.676]	54.817/0.728; 75.533 [57.803–82.877]	18.035/0.476; 37.928 [35.338–41.236]
Triangle RT-2A1	11847.933/3508.170; 3.411 [3.292–3.591]	3516.714/1312.973; 2.732 [2.449–2.806]	602.856/437.096; 1.379 [1.300–1.415]
LibRTS point	5502.395/4147.360; 1.312 [1.061–2.021]	10.796/0.902; 12.059 [9.440–17.776]	8.916/0.230; 38.467 [34.687–41.240]
LibRTS range	4565.787/4173.109; 1.080 [1.009–1.196]	12.352/0.890; 13.422 [12.182–23.731]	9.140/0.233; 39.083 [34.129–42.385]

8 Related Work

8.1 Repurposed RT applications and abstractions

A scoped recent review reports 35 proposed RT solutions across 32 problems and already identifies coherent design of geometry, queries, and hit-produced operations as a recurring concern [19]. RayJoin, RayDB, RTNN, RT-DBSCAN, graph triangle counting, X-HD, particle tracking, and RT-BarnesHut each supply application-specific mappings and result handling [6, 8, 21, 22, 29, 34, 35, 37]. They establish both the opportunity and prior solutions to individual obligations; RTDL does not claim to discover geometric encoding, all-hit continuation, deduplication, nearest-K maintenance, or graph orientation.

LibRTS is the closest application-facing abstraction in this set. It provides point and range spatial queries plus user device handlers for counting and collection [7]. RTDL is narrower in supported operations and broader in a different dimension: it accepts restricted role source, represents typed effects and result-route constraints, and binds the accepted program to generated target and runtime identities. We have not run a head-to-head user study, and do not claim superiority or that LibRTS cannot add such checks.

TTA/TTA+ instead changes RT hardware and its interface to support more general tree traversal [10]. RTDL works with existing OptiX hardware after the author has selected an RT mapping; it neither removes fixed-function limits nor finds a profitable mapping automatically.

8.2 RT languages and compilation systems

OptiX established programmable RT, including early termination, continued intersection processing, attributes, and non-rendering examples [27]. Current OptiX, Vulkan, and DXR specify substantial pipeline, interface, payload, and SBT rules [16, 17, 20, 24]. PyOptiX exposes the OptiX host API to Python, while OWL owns much pipeline construction [25, 26]. These are not unchecked baselines.

Shader Components and Slang provide modular interfaces, specialization, and reflection; Slang capabilities validate target, stage, API, and hardware requirements, and shader cursors address parameter layout and binding [11, 12, 30, 31]. SlangPy adds Python RT pipelines, hit groups, shader tables, dispatch, marshalling, caching, and lifecycle support [15, 32]. Luisa automates dependency analysis and RT pipeline/SBT/launch work; CrossRT generates host/device and RT mappings from restricted C++; Dr.Jit traces and specializes broad Python/C++ computation through OptiX [4, 14, 18, 36]. Scion is a DSL/compiler that separates BVH logical trees, physical layouts, and traversal algorithms with static well-formedness checks [9]. These systems preclude claims that RTDL first automates RT construction, relates logic to physical structure, or specializes code from result use.

Rendering DSLs also encode domain contracts. Open Shading Language separates shader-produced closures from renderer-owned integration, and shader-component systems coordinate code with required parameters [1, 12]. RTDL applies a bounded language design to selected non-rendering result protocols; it does not claim that rendering lacks cross-stage semantics.

The concrete RTDL increment is its implemented connection, for supported fixed families, from declared result obligations to role-effect admission, semantic and physical contracts, trusted traversal-action generation, executable identity, and interface-specific failure checks. It uses explicit family rules and guarded specialization at the cost of restricted expressiveness and topology-specific trusted code. The available sources for adjacent systems do not establish this exact combination, but source silence is not an inability result and those systems may be extensible.

8.3 Protocol and validation foundations

Typestate, interface automata, and multiparty session types provide general and often formally stronger models of state and global/local compatibility [3, 13, 33]. Typed linking and FFI checking already relate cross-language behavior, representations, offsets, tags, and effects [5, 28]. Proof-carrying code established consumer-side policy validation of supplied proofs before native execution [23]. RTDL contributes neither a new linking calculus nor a proof certificate. Schemas, projections, lowerers, and runtime remain trusted; hashes establish identity, not semantics. Its finite target checker is empirical and structural, unlike formal GPU verification such as GPUVerify [2].

9 Artifact

The nine-member evidence-only archive contains an anonymous projection and standard-library verifier over 160

formal cells, 20,480 steady samples, 1,024 A-only instrumentation endpoints, 20 AOT samples, and eight competence records. Two clean exports were byte-identical and replayed in normal and optimized Python from fresh directories. It bundles no third-party code, raw GPU archive, or CUDA/OptiX material and performs offline recount, not GPU rerun, installation, or private-history reconstruction.

The application transaction is retained separately from this nine-member archive. Its worker records support offline statistical recount, but it does not include all raw-domain inputs or rerun application oracles.

10 Threats to Validity

Construct validity. Prepared A/D excludes authoring, cold compilation, and first use. Direct D is a named contract-equivalent implementation, not a proven latency lower bound; strong C is output-equivalent but not operation-identical. These observations do not isolate intrinsic language overhead, and paired instrumentation covers A only. Application complete starts after domain-encoding loaders and first execute after preparation; compilation, physical routes, and timed validation differ, with no causal ablation. The same-width semantic-ABI example is illustrative: mutation tests establish check liveness, but no retained run executes its defective route. A trusted semantic schema may also be wrong while internally coherent.

Internal validity. Both synthetic tasks were adapted, not unseen. Entry was revised after an adverse result; both first-result endpoints remain lifecycle-confounded and are not reinterpreted as passes. Mocks show that a provider double fault can replace the primary exception and lose retry ownership, while a native fork can evade the cached-PID guard. Neither mock ran GPU work or appeared in retained workers.

External validity. Synthetic measurements cover two NVIDIA generations and two exact tasks without cross-machine raw-time comparison. The application study covers selected stages from 3/9 apps, four units, and one RTX A4500; all twelve rows are adverse, six apps are unexecuted, and inputs are derived representations. Authors defined the challenge domain and selected one composition. There is no independent-user, prevalence, or arbitrary-Python/IR evidence and no first/only/impossibility claim. Although all nine entry points were recovered, six lack retained historical per-file identities and the original archive; older V2/V4 timings are not pooled with M.

Implementation trust. The sphere route required about 2,635 topology-specific TCB lines; the finite checker missed an early-return probe; 4,096 timed A calls lack per-call receipts.

Artifact scope. The derived anonymous projection checks identities, counts, formulas, and rows, but cannot recreate GPU runs or private custody.

11 Conclusion

RTDL is a restricted-Python DSL and compiler for bounded repurposed ray tracing. Authors express typed data and role behavior while retaining ownership of the application mapping and semantic oracle. Typed Callback IR and whole-protocol admission connect selected result obligations to role effects, semantic and physical contracts, continuation, and executable identity. Generated Numba leaves and trusted topology wrappers implement supported authored templates; closed routes can select dedicated native implementations, including exact-IR specializations for the measured standard routes.

The design produces concrete compiler behavior: it rejects a role-legal effect that conflicts with one complete-relation route, generates physical intersection ignore for logically accepted all-hit events, and exits a fixed specialization when source semantics change. Nine historical V4 mappings show where these components compose and where application responsibility remains; they do not prove arbitrary generalization or usability. Across two exact tasks and two GPU generations, prepared median latency is 1.077–1.175× Direct, while post-import comparison reaches 1.837× median and a 2.377× worst block. Selected-stage application V4/PyOptiX is adverse at all twelve endpoints, including 1.379× graph prepared and 37.928–39.083× Particle/LibRTS prepared. This establishes bounded functional feasibility while exposing route/lifecycle debt, not broad practical performance or an authoring-productivity advantage.

References

[1] Academy Software Foundation. 2026. Open Shading Language Documentation. https://open-shading-language.readthedocs.io/en/latest/. Programmable shading for advanced renderers.

[2] Adam Betts, Nathan Chong, Alastair F. Donaldson, Shaz Qadeer, and Paul Thomson. 2012. GPUVerify: A Verifier for GPU Kernels. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications*. ACM, New York, NY, USA, 113–132. https://doi.org/10.1145/2384616.2384625

[3] Luca de Alfaro and Thomas A. Henzinger. 2001. Interface Automata. In *Proceedings of the 8th European Software Engineering Conference Held Jointly with the 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 109–120. https://doi.org/10.1145/503209.503226

[4] Vladimir Frolov, Vadim Sanzharov, Albert Garifullin, Maxim Raenchuk, and Alexei Voloboy. 2024. CrossRT: A Cross Platform Programming Technology for Hardware-Accelerated Ray Tracing in CG and CV Applications. arXiv:2409.12617. https://doi.org/10.48550/arXiv.2409.12617

[5] Michael Furr and Jeffrey S. Foster. 2005. Checking Type Safety of Foreign Function Calls. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, New York, NY, USA, 62–72. https://doi.org/10.1145/1065010.1065019

[6] Liang Geng, Rubao Lee, and Xiaodong Zhang. 2024. RayJoin: Fast and Precise Spatial Join. In *Proceedings of the 38th ACM International Conference on Supercomputing*. 124–136. https://doi.org/10.1145/3650200.3656610

[7] Liang Geng, Rubao Lee, and Xiaodong Zhang. 2025. LibRTS: A Spatial Indexing Library by Ray Tracing. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 396–411. https://doi.org/10.1145/3710848.3710850

[8] Liang Geng, Zhehu Yuan, Rubao Lee, Fusheng Wang, and Xiaodong Zhang. 2026. X-HD: Fast Hausdorff Distance Computation with Ray Tracing. In *Proceedings of the 40th ACM International Conference on Supercomputing*. 945–959. https://doi.org/10.1145/3797905.3800509

[9] Christophe Gyurgyik, Alexander J. Root, and Fredrik Kjolstad. 2026. Decoupling Data Layouts from Bounding Volume Hierarchies. *Proceedings of the ACM on Programming Languages* 10, PLDI, Article 175 (2026), 39 pages. https://doi.org/10.1145/3808253

[10] Dongho Ha, Lufei Liu, Yuan Hsi Chou, Seokjin Go, Won Woo Ro, Hung-Wei Tseng, and Tor M. Aamodt. 2024. Generalizing Ray Tracing Accelerators for Tree Traversals on GPUs. In *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture*. 1041–1057. https://doi.org/10.1109/MICRO61859.2024.00080

[11] Yong He, Kayvon Fatahalian, and Theresa Foley. 2018. Slang: Language Mechanisms for Extensible Real-Time Shading Systems. *ACM Transactions on Graphics* 37, 4, Article 141 (2018), 13 pages. https://doi.org/10.1145/3197517.3201380

[12] Yong He, Theresa Foley, Teguh Hofstee, Haomin Long, and Kayvon Fatahalian. 2017. Shader Components: Modular and High Performance Shader Development. *ACM Transactions on Graphics* 36, 4, Article 100 (2017), 11 pages. https://doi.org/10.1145/3072959.3073648

[13] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2016. Multiparty Asynchronous Session Types. *J. ACM* 63, 1, Article 9 (2016), 67 pages. https://doi.org/10.1145/2827695

[14] Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, and Delio Vicini. 2022. Dr.Jit: A Just-In-Time Compiler for Differentiable Rendering. *ACM Transactions on Graphics* 41, 4 (2022), 1–19. https://doi.org/10.1145/3528223.3530099

[15] Simon Kallweit, Chris Cummings, Benedikt Bitterli, Sai Bangaru, and Yong He. 2026. SlangPy v0.43.1. https://github.com/shader-slang/slangpy/releases/tag/v0.43.1. Release source commit 2f6c4625fdd2b3bd812ca6cd2802cf98bd89b248.

[16] Khronos Group. 2026. Vulkan Specification: Ray Tracing and Shader Binding Tables. https://docs.vulkan.org/spec/latest/chapters/raytracing.html.

[17] Khronos Group. 2026. Vulkan Specification: Shader Interfaces. https://docs.vulkan.org/spec/latest/chapters/interfaces.html.

[18] LuisaGroup. 2026. LuisaCompute: High-Performance Rendering Framework on Stream Architectures. https://github.com/LuisaGroup/LuisaCompute.

[19] Enzo Meneses, Cristóbal A. Navarro, Héctor Ferrada, Konstantin Verichev, and Cristian Salazar-Concha. 2027. Ray Tracing Cores for General-Purpose Computing: A Literature Review. *Future Generation Computer Systems* 186, Article 108739 (2027). https://doi.org/10.1016/j.future.2026.108739 Available online 3 August 2026.

[20] Microsoft. 2026. DirectX Raytracing Functional Specification: Payload Access Qualifiers. https://microsoft.github.io/DirectX-Specs/d3d/Raytracing.html.

[21] Vani Nagarajan, Rohan Gangaraju, Kirshanthan Sundararajah, Artem Pelenitsyn, and Milind Kulkarni. 2025. RT-BarnesHut: Accelerating Barnes–Hut Using Ray-Tracing Hardware. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 43–56. https://doi.org/10.1145/3710848.3710885

[22] Vani Nagarajan and Milind Kulkarni. 2023. RT-DBSCAN: Accelerating DBSCAN Using Ray Tracing Hardware. In *2023 IEEE International Parallel and Distributed Processing Symposium*. 963–973. https://doi.org/10.1109/IPDPS54959.2023.00100

[23] George C. Necula and Peter Lee. 1996. Safe Kernel Extensions Without Run-Time Checking. In *Proceedings of the 2nd USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Seattle, WA, 229–243. https://www.usenix.org/conference/osdi-96/safe-kernel-extensions-without-run-time-checking

[24] NVIDIA. 2025. *NVIDIA OptiX 9.0 Programming Guide*. NVIDIA. https://raytracing-docs.nvidia.com/optix9/guide/optix_guide.250130.A4.pdf Payload types and per-stage payload semantics.

[25] NVIDIA. 2026. PyOptiX: Complete Python Bindings for the OptiX Host API. https://github.com/NVIDIA/otk-pyoptix. Pinned experiment source commit 3144f224c0fd18733925faf3d8fb82c7376b8dcf.

[26] NVIDIA, Ingo Wald, Stefan Zellmann, and contributors. 2026. OWL: The OptiX Wrappers Library. https://github.com/NVIDIA/OWL.

[27] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: A General Purpose Ray Tracing Engine. *ACM Transactions on Graphics* 29, 4, Article 66 (2010), 13 pages. https://doi.org/10.1145/1778765.1778803

[28] Daniel Patterson and Amal Ahmed. 2017. Linking Types for Multi-Language Software: Have Your Cake and Eat It Too. In *2nd Summit on Advances in Programming Languages (Leibniz International Proceedings in Informatics, Vol. 71)*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 12:1–12:15. https://doi.org/10.4230/LIPIcs.SNAPL.2017.12

[29] Xuri Shi, Kai Zhang, X. Sean Wang, Xiaodong Zhang, and Rubao Lee. 2025. RayDB: Building Databases with Ray Tracing Cores. *Proceedings of the VLDB Endowment* 19, 1 (2025), 43–55. https://doi.org/10.14778/3772181.3772185

[30] Slang Project. 2026. Capabilities—Slang User's Guide. https://docs.shader-slang.org/en/stable/external/slang/docs/user-guide/05-capabilities.html.

[31] Slang Project. 2026. A Practical and Scalable Approach to Cross-Platform Shader Parameter Passing Using Slang's Reflection API. https://docs.shader-slang.org/en/stable/shader-cursors.html.

[32] SlangPy Project. 2026. SlangPy API Reference and Changelog. https://slangpy.shader-slang.org/en/stable/src/api_reference.html.

[33] Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171. https://doi.org/10.1109/TSE.1986.6312929

[34] Bin Wang, Ingo Wald, Nate Morrical, Will Usher, Lin Mu, Karsten Thompson, and Richard Hughes. 2022. An GPU-Accelerated Particle Tracking Method for Eulerian–Lagrangian Simulations Using Hardware Ray Tracing Cores. *Computer Physics Communications* 271 (2022), 108221. https://doi.org/10.1016/j.cpc.2021.108221

[35] Zhixiong Xiao, Mengbai Xiao, Yuan Yuan, Dongxiao Yu, Rubao Lee, and Xiaodong Zhang. 2025. A Case Study for Ray Tracing Cores: Performance Insights with Breadth-First Search and Triangle Counting in Graphs. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 2, Article 16 (2025), 25 pages. https://doi.org/10.1145/3727108

[36] Shaokun Zheng, Zhiqian Zhou, Xin Chen, Difei Yan, Chuyan Zhang, Yuefeng Geng, Yan Gu, and Kun Xu. 2022. LuisaRender: A High-Performance Rendering Framework with Layered and Unified Interfaces on Stream Architectures. *ACM Transactions on Graphics* 41, 6, Article 232 (2022), 19 pages. https://doi.org/10.1145/3550454.3555463

[37] Yuhao Zhu. 2022. RTNN: Accelerating Neighbor Search Using Hardware Ray Tracing. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 76–89. https://doi.org/10.1145/3503221.3508409